K.R. MANGALAM UNIVERSITY

THE COMPLETE WORLD OF EDUCATION

Fundamentals Of Operating System Lab

(ENCA252)

Lab File

Submitted by: Mayank Chauhan (2401201136)

Submitted to

Ms. Jyoti Yadav

School of Engineering & Technology

OPERATING SYSTEM LAB – ASSIGNMENT 1

Topic: Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

---

1. Aim

To implement First Come First Serve (FCFS) and Shortest Job First (SJF) scheduling algorithms and analyze their performance using parameters such as Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

---

2. Theory

CPU Scheduling is a process by which the operating system decides which process will execute next. It helps in efficient utilization of CPU.

FCFS (First Come First Serve):

- Processes are executed in the order of their arrival time.
- It is a non-preemptive scheduling algorithm.
- Simple but may lead to higher waiting time.

SJF (Shortest Job First):

- Process with the smallest burst time is executed first.
- It minimizes average waiting time.
- Also non-preemptive in this implementation.

---

3. Algorithm

FCFS Algorithm:

1. Sort processes based on arrival time.

2. Execute each process in order.

3. Calculate CT, TAT = CT − AT, WT = TAT − BT.

SJF Algorithm:

1. Select processes that have arrived.

2. Choose process with minimum burst time.

3. Execute and repeat until all processes complete.

---

4. Code

```python
class Process:
    def __init__(self, pid, at, bt):
        self.pid = pid
        self.at = at
        self.bt = bt
        self.ct = 0
        self.tat = 0
        self.wt = 0


def print_table(processes):
    print("\nPID\tAT\tBT\tCT\tTAT\tWT")
    for p in processes:
        print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")


# ================= FCFS =================
def fcfs(processes):
    processes.sort(key=lambda x: x.at)
    time = 0
    gantt = []

    for p in processes:
        if time < p.at:
            gantt.append(("Idle", time, p.at))
            time = p.at

        start = time
        time += p.bt
        p.ct = time
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt

        gantt.append((p.pid, start, time))

    print("\n--- FCFS ---")
    print_table(processes)

    avg_wt = sum(p.wt for p in processes) / len(processes)
    avg_tat = sum(p.tat for p in processes) / len(processes)

    print(f"\nAverage WT = {avg_wt:.2f}")
    print(f"Average TAT = {avg_tat:.2f}")

    print("\nGantt Chart:")
    for g in gantt:
        print(f"| {g[0]} ", end="")
    print("|")

    for g in gantt:
        print(f"{g[1]}\t", end="")
    print(f"{gantt[-1][2]}")


# ================= SJF =================
def sjf(processes):
    processes = sorted(processes, key=lambda x: (x.at, x.bt))
    completed = []
    time = 0
    ready = []
    gantt = []

    processes_copy = processes[:]

    while processes_copy or ready:
        while processes_copy and processes_copy[0].at <= time:
            ready.append(processes_copy.pop(0))

        if not ready:
            time += 1
            continue

        ready.sort(key=lambda x: x.bt)
        p = ready.pop(0)

        start = time
        time += p.bt

        p.ct = time
        p.tat = p.ct - p.at
        p.wt = p.tat - p.bt

        gantt.append((p.pid, start, time))
        completed.append(p)

    print("\n--- SJF (Non-Preemptive) ---")
    print_table(completed)

    avg_wt = sum(p.wt for p in completed) / len(completed)
    avg_tat = sum(p.tat for p in completed) / len(completed)

    print(f"\nAverage WT = {avg_wt:.2f}")
    print(f"Average TAT = {avg_tat:.2f}")

    print("\nGantt Chart:")
    for g in gantt:
        print(f"| {g[0]} ", end="")
    print("|")

    for g in gantt:
        print(f"{g[1]}\t", end="")
    print(f"{gantt[-1][2]}")


# ================= MAIN =================
n = int(input("Enter number of processes: "))
processes = []

for i in range(n):
    at = int(input(f"Enter Arrival Time for P{i+1}: "))
    bt = int(input(f"Enter Burst Time for P{i+1}: "))
    processes.append(Process(f"P{i+1}", at, bt))

# Copy list for separate execution
import copy
fcfs(copy.deepcopy(processes))
sjf(copy.deepcopy(processes))
```

---

5. Input Used

Example Input:

Number of Processes = 5

Process	AT	BT
P1	0	8
P2	1	4
P3	2	9
P4	3	5
P5	4	2

---

6. Output

```
Enter number of processes: 5
Enter Arrival Time for P1: 7
Enter Burst Time for P1: 8
Enter Arrival Time for P2: 4
Enter Burst Time for P2: 2
Enter Arrival Time for P3: 8
Enter Burst Time for P3: 6
Enter Arrival Time for P4: 5
Enter Burst Time for P4: 9
Enter Arrival Time for P5: 1
Enter Burst Time for P5: 2

--- FCFS ---

PID     AT      BT      CT      TAT     WT
P5      1       2       3       2       0
P2      4       2       6       2       0
P4      5       9       15      10      1
P1      7       8       23      16      8
P3      8       6       29      21      15

Average WT = 4.80
Average TAT = 10.20

Gantt Chart:
| Idle | P5 | Idle | P2 | P4 | P1 | P3 |
0       1       3       4       6       15      23      29

--- SJF (Non-Preemptive) ---

PID     AT      BT      CT      TAT     WT
P5      1       2       3       2       0
P2      4       2       6       2       0
P4      5       9       15      10      1
P3      8       6       21      13      7
P1      7       8       29      22      14

Average WT = 4.40
Average TAT = 9.80

Gantt Chart:
| P5 | P2 | P4 | P3 | P1 |
1       4       6       15      21      29
```

- FCFS Output Table
- SJF Output Table
- Gantt Charts

---

7. Result

- FCFS executes processes in arrival order, which may lead to higher waiting time.
- SJF selects the shortest job first, reducing the average waiting time.
- From the results, it is observed that SJF performs better than FCFS in terms of efficiency.

---

8. Conclusion

CPU scheduling plays an important role in process management.

Among the two algorithms implemented, SJF provides better performance by minimizing waiting time, while FCFS is simpler but less efficient.

---

9. GitHub Link

(https://github.com/MayankCh-Work/os-lab-assignments.git)

```python
            if not ready:
                time += 1
                continue

            ready.sort(key=lambda x: x.bt)
            p = ready.pop(0)

            start = time
            time += p.bt

            p.ct = time
            p.tat = p.ct - p.at
            p.wt = p.tat - p.bt

            gantt.append((p.pid, start, time))
            completed.append(p)

        print("\n--- SJF (Non-Preemptive) ---")
        print_table(completed)

        avg_wt = sum(p.wt for p in completed) / len(completed)
        avg_tat = sum(p.tat for p in completed) / len(completed)

        print(f"\nAverage WT = {avg_wt:.2f}")
        print(f"Average TAT = {avg_tat:.2f}")

        print("\nGantt Chart:")
        for g in gantt:
            print(f"| {g[0]} ", end="")
        print("|")

        for g in gantt:
            print(f"{g[1]}\t", end="")
        print(f"{gantt[-1][2]}")


    # ================= MAIN ================
    n = int(input("Enter number of processes: "))
    processes = []

    for i in range(n):
        at = int(input(f"Enter Arrival Time for P{i+1}: "))
        bt = int(input(f"Enter Burst Time for P{i+1}: "))
        processes.append(Process(f"P{i+1}", at, bt))

    # Copy list for separate execution
    import copy
    fcfs(copy.deepcopy(processes))
    sjf(copy.deepcopy(processes))
```